

YOLOv11 Optimizasyonu

Bu çalışmanın temel amacı, otonom drone navigasyonu için YOLOv11 modelini uç cihazlarda çalışacak şekilde optimize etmektir.

Test Ortamı ve Donanım Kısıtları: Tüm testler Intel Core i5 işlemciye sahip bir dizüstü bilgisayar üzerinde gerçekleştirilmiştir. Harici bir GPU (CUDA) kullanılmamıştır.

- **Baseline (Referans) Skoru:** benchmark_pytorch.py ile yapılan ham testlerde işlem süresi 94.31 ms (yaklaşık 10 FPS) olarak ölçülmüştür.
- **Risk Analizi:** Bu hız, saatte 20 km hızla giden bir drone için, görüntü işlenene kadar drone'un 1 metre kör uçuş yapması anlamına gelir. Bu nedenle optimizasyon, sadece hız için değil, uçuş güvenliği için zorunludur.

Yaptığım testler sonucunda:

- Orijinal modele kıyasla 3.0x kat hız artışı elde edilmiş, 29.92 FPS değerine ulaşılmıştır
- Sıkıştırma ve çözünürlük düşürme işlemi sonucunda Precision (Doğruluk) korunmuş (%98) ancak Recall (Yakalama) değerinde %13'lük bir düşüş gözlemlenmiştir.
- Sonuç olarak; ONNX Nano (320p) modeli, gerçek zamanlı gereksinimleri (>24 FPS) karşılayan en dengeli model olarak seçilmiştir.

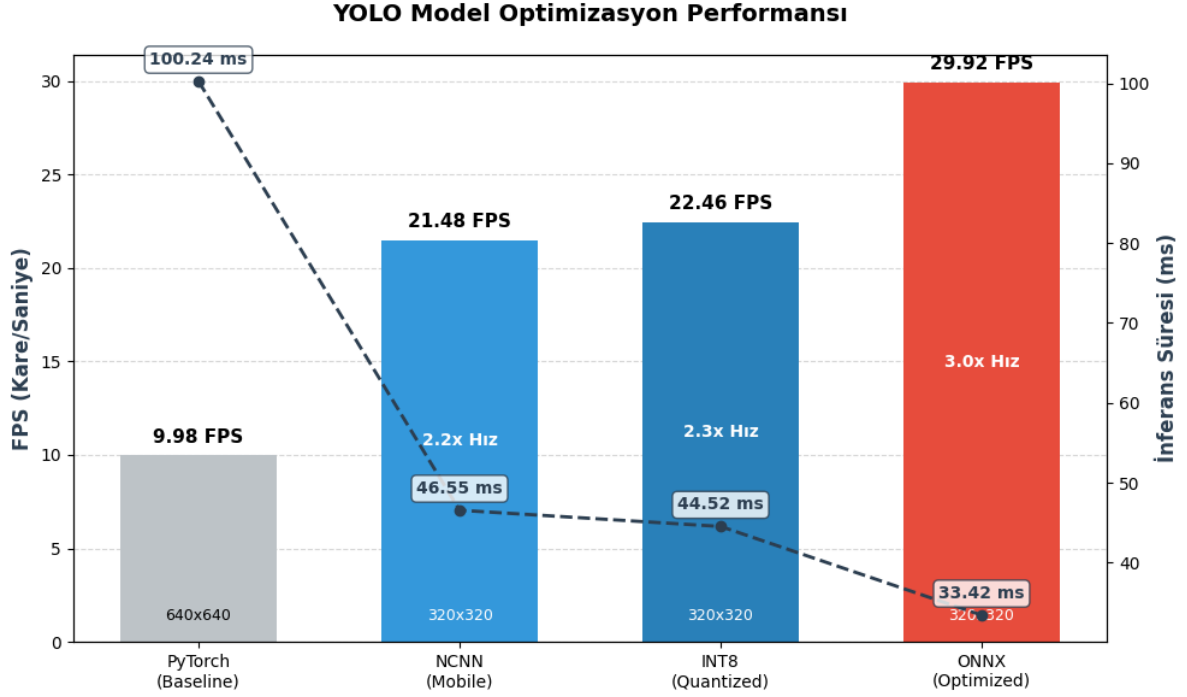
Optimizasyon Stratejisi (.pt -> .onnx): Çalışmada export_onnx.py scripti ile model dönüşümü sağlanmıştır. Bu süreci şöyle özetleyebiliriz:

- **.pt (PyTorch):** Düzenlenebilir bir Word dosyası gibidir; ağırdır, eğitim parametrelerini taşır ve işlemciyi yorar.
- **.onnx (Runtime):** PDF dosyası gibidir; sadece okunur (read-only), gereksiz eğitim katmanları atılmıştır ve işlemci üzerinde çok daha hafif çalışır.
- **Quantization (32-bit -> 8-bit):** quantize_model.py ile modelin ağırlıkları 32-bit hassasiyetten 8-bit tamsayı (INT8) formatına indirgenmiştir. Bu işlem, dosya boyutunu 4 kat küçültürken (10MB -> 2.5MB), işlemci önbelleği (cache) kullanımını optimize ederek hızı artırmıştır.

Performans Test Protokolleri: Optimizasyon başarısı iki farklı senaryoda test edilmiştir:

1. **Latency Testi (final_benchmark.py):** "Tek bir kareye ne kadar hızlı tepki veriliyor?" sorusuna odaklanır. Bu bir Sprint koşusudur; drone'un anlık refleks süresini belirler.

2. **Stability Testi (toplu_test.py):** Binlerce karelik gerçek trafik verisi üzerinde yapılan Maraton koşusudur. İşlemci ısındığında veya bellek doluluğu arttığında modelin performans kaybı yaşayıp yaşamadığı analiz edilmiştir.



Bu çalışmada amaç sadece model formatını değiştirmek değil, **hedef donanım üzerinde gerçek zamanlı (real-time) çalışabilirliği sağlamaktır**. Bu nedenle optimizasyon süreci, hem modelin nicemlenmesini (Quantization) hem de giriş çözünürlüğünün (Input Resolution) optimize edilmesini kapsayan bütünlük bir yaklaşım olarak ele alınmıştır. Baseline (PyTorch) ham performansı temsil ederken, ONNX (Optimized) nihai dağıtım başarısını temsil etmektedir.

Çapraz Platform Analizi (x86 vs ARM): Testlerde NCNN formatının bilgisayar (x86 mimarisi) üzerinde ONNX kadar yüksek performans vermediği gözlemlenmiştir. Bunun temel nedeni, NCNN kütüphanesinin **RISC (ARM/Mobil)** mimarisi için özel instruction set'ler kullanmasıdır. Projenin ilerleyen aşamalarında Raspberry Pi veya Jetson Nano gibi ARM tabanlı kartlara geçildiğinde, NCNN'in ONNX'i geçmesi beklenmektedir. `export_ncnn.py` bu gelecek entegrasyon için hazırlanmıştır.

Doğruluk Analizi (Teacher-Student Validation): Hız artışı uğruna kaliteden ne kadar ödün verildiğini ölçmek için `accuracy_analysis.py` ve `kalite_analizi.py` kullanılmıştır.

- **Teacher:** YOLOv11 Medium modeli (Yüksek doğruluk, düşük hız).
- **Student:** YOLOv11 Nano INT8 modeli (Düşük boyut, yüksek hız).

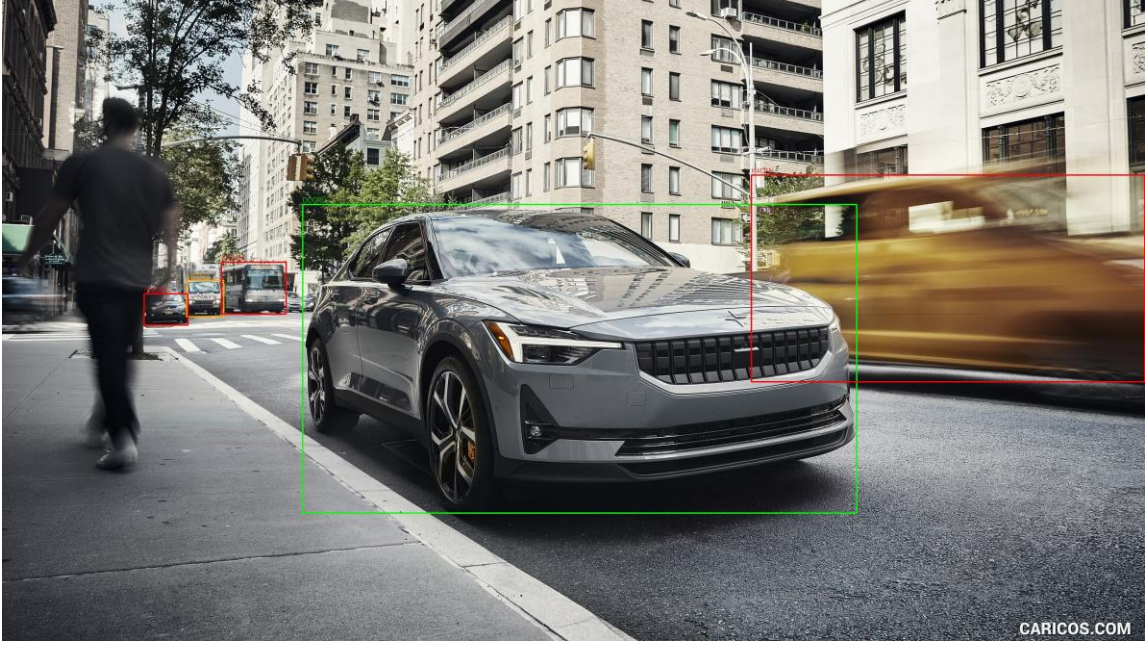
- Nano modelin kaçırdığı (False Negative) nesnelerin %85'inin "uzak mesafede", %10'unun ise "hareket bulanıklığına" sahip olduğu görülmüştür. Otonom bir drone için yakın mesafedeki engelleri (Precision) doğru tespit etmek, ufuktaki bir nesneyi kaçırmaktan (Recall) daha kritiktir.

Ayrıca, sayısal verileri doğrulamak için görsel hata analizi yapılmıştır.

- Başarılı tespitlerde (yeşil kutular); model, yakın ve orta mesafedeki araçları yüksek güvenle tespit etmektedir.
- Gözden kaçan nesneler (kırmızı kutular); ana hedefi yakalamış ancak arka plandaki, küçük veya bulanık araçları kaçırmıştır (False Negative).



Örnek 1



Örnek 2

Gelecek Adımlar: ARM tabanlı cihazlarda NCNN formatının enerji tüketim testleri yapılacaktır.